

Lab 4: VibeCoding — Chat History

Repository: 51_rhythm-tap

Student: Suhas Mohan Kumar

Vibe Coding interaction used to implement the four feature tasks specified in the repository README.

1. Task 1 — Sound Effects on Hit

USER:

I am working on the Python Pygame rhythm game in the assigned repository. Please implement ONLY Task 1 from the README: Sound Effects on Hit.

Requirements:

- Play a short beep or drum-like sound whenever the player successfully receives a PERFECT, GREAT, or OK judgement.
- Do not play a sound when the result is MISS.
- Use Pygame's existing mixer/audio functionality.
- If practical, generate a simple sound programmatically so that no external audio asset is required.
- Keep the existing gameplay mechanics, scoring system, controls, and visuals unchanged.
- Do not implement any of the other README tasks yet.
- Make the implementation clean and robust and avoid unnecessary changes to unrelated files.
- Briefly identify the files modified and explain the implementation.
- Verify that the game still runs with ``python main.py``.

ASSISTANT:

Task 1 was implemented in `game/game_engine.py`. A short beep was generated programmatically using Python's `array/math` modules and Pygame's mixer. The sound is played only on successful PERFECT, GREAT, or OK hits, and not on MISS. The rest of the gameplay, scoring, controls, and visuals were kept unchanged.

ASSISTANT:

Commit after testing:

```
git add game/game_engine.py
git commit -m "Add hit sound effects"
git push
```

2. Task 2 — Hold Notes

USER:

Now implement ONLY Task 2 from the README: Hold Notes. Please build this on top of the current Task 1 implementation without removing or breaking the hit sound functionality.

Requirements:

- Add a new type of long/hold note.
- The player should need to press and continuously hold the corresponding lane key for 1 second.
- Releasing the key before the required duration should result in a MISS.
- Successfully completing the hold should count as a successful hit and provide appropriate feedback/score.
- Display the hold note clearly as a longer note so the player can distinguish it from a normal note.
- Keep normal notes working as before.
- Do not implement Task 3 or Task 4 yet.
- Minimize changes outside the files necessary for this feature.
- Make sure the game still runs with ``python main.py``.

ASSISTANT:

Task 2 adds hold notes while keeping Task 1 working. Hold notes have a longer visual body and require the player to keep the corresponding lane key pressed for 1 second. Releasing early causes a MISS; successfully completing the hold awards points and plays the hit sound.

The implementation uses:

- `is_hold`
- `hold_duration`
- `holding`
- `hold_start_time`

A random hold note is spawned with a 25% probability, with a required hold duration of 1000 ms.

ASSISTANT:

Commit after testing:

```
git add game/beat.py game/game_engine.py
```

```
git commit -m "Add hold notes"
```

```
git push
```

3. Task 3 — BPM-Synced Spawning

USER:

Please implement ONLY Task 3 from the README: BPM-Synced Spawning, while preserving the functionality from Tasks 1 and 2.

The current game uses a fixed frame-based spawn interval. Replace that spawning mechanism with a beat-based system controlled by a configurable BPM value.

Requirements:

- Use a BPM value of 120 as the default.
- Calculate the interval between beats from the BPM rather than relying on the old arbitrary spawn interval.
- At 120 BPM, notes should be generated once every 0.5 seconds.
- Prevent timing drift by advancing the next beat from the previous scheduled beat rather than resetting it from the current time.
- Keep hold notes, hit sounds, scoring, controls, and existing visuals working.
- The gradual increase in note movement speed may remain, but it must not determine when notes spawn.
- Do not implement Task 4 yet.
- Keep the implementation clean and limited to the files needed.
- Verify the game still runs with ``python main.py``.

ASSISTANT:

Task 3 was implemented by replacing the old frame-based spawn interval with a BPM-derived beat interval.

The implementation uses:

`BPM = 120`

`BEAT_INTERVAL = 60000 / BPM`

At 120 BPM, one beat occurs every 500 ms. The game tracks the next beat time and spawns a note whenever the current time reaches that beat.

ASSISTANT:

To demonstrate that BPM controls spawning, the BPM can temporarily be changed:

`BPM = 60` → one beat every 1 second

`BPM = 120` → one beat every 0.5 seconds

A temporary beat counter/print can also be used for verification, then removed before the final commit.

ASSISTANT:

Commit:

```
git add game/game_engine.py
```

```
git commit -m "Add BPM-synced note spawning"
```

```
git push
```

4. Task 4 — Grade Summary Screen

USER:

Please implement ONLY Task 4 from the README: Grade Summary Screen. This should be added on top of the existing Tasks 1–3 implementation without changing their behavior.

Requirements:

- Track the number of PERFECT, GREAT, OK, and MISS results throughout a game.
- When the game ends, display a clear grade summary screen.
- Show the count for each judgement category.
- Calculate and display the player's accuracy percentage.
- Accuracy should represent successful judgements (PERFECT + GREAT + OK) divided by the total number of judgements, including MISS.
- Also retain useful existing information such as final score and maximum combo.
- Provide a restart option using the existing R control.
- Do not remove or break sound effects, hold notes, or BPM-synchronized spawning.
- Keep the implementation consistent with the existing Pygame interface and avoid unnecessary changes.

ASSISTANT:

Task 4 adds a Grade Summary screen while preserving Tasks 1–3.

The game now keeps counters for:

- PERFECT
- GREAT
- OK
- MISS

The accuracy calculation is:

$$(\text{PERFECT} + \text{GREAT} + \text{OK}) / \text{TOTAL} \times 100$$

The grade summary is displayed when the game reaches GAME OVER and shows the judgement counts, accuracy percentage, final score, maximum combo, and the existing restart instruction.

ASSISTANT:

Commit:

```
git add game/game_engine.py
```

```
git commit -m "Add grade summary screen"
```

```
git push
```

5. Final Status

ASSISTANT:

Completed:

- Task 1 — Sound Effects on Hit
- Task 2 — Hold Notes
- Task 3 — BPM-Synced Spawning
- Task 4 — Grade Summary Screen

Remaining submission items:

- Final 10-second after video
- Exported chat history as a PDF/document
- Upload before/after videos, updated code, and chat history under the Lab-4 folder in the Lab repository.